

SOFIA UNIVERSITY
ST. KLIMENT OHRIDSKI



Проект по СПО

Паралелно пресмятане на числото на Ойлер (e) чрез конвейерна архитектура

Гаврил Христов
ФН: 6MI0800460
спец. Компютърни Науки
курс 3, втори поток, група 7

2026

Съдържание

1	Увод	2
1.1	Развитие на числото e в ред на Тейлър	2
1.2	Цел на проекта	2
2	Анализ на алгоритъма	2
2.1	Метод за паралелно пресмятане на факториели и суми	2
2.2	Метод за балансиране на задачите	2
2.3	Сложност на аритметиката с произволна точност	3
2.4	Последователност на изпълнение на алгоритъма	3
2.5	Функционален анализ	4
2.6	Нефункционален анализ (Технологичен анализ)	5
3	Проектиране и реализация	5
3.1	Функционално проектиране	5
3.1.1	Описание на функциите и коментар на алгоритмите	5
3.1.2	Диаграми	6
3.1.3	Потребителски интерфейси и екрани на реакция	6
3.1.4	Ръководство за потребителя (Параметри)	8
3.2	Технологично проектиране	8
4	Тестване	8
4.1	Методология на тестване	8
4.2	Резултати	9
4.3	Анализ и обобщение	15
5	Източници	16

1 Увод

1.1 Развитие на числото e в ред на Тейлър

Числото e притежава уникалното свойство да бъде основа на естествената експоненциална функция. Математическата дефиниция чрез ред на Тейлър е:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \dots$$

Този ред осигурява бърза сходимост, тъй като факториелът в знаменателя нараства свръхекспоненциално.

1.2 Цел на проекта

Основната цел на проекта е проектирането и реализацията на паралелна система за изчисляване на числото e с висока точност. Системата използва конвейерна архитектура за разпределяне на изчисленията, с цел постигане на оптимална скалируемост чрез балансирано натоварване на нишките спрямо сложността на изчисление на факториелите.

2 Анализ на алгоритъма

2.1 Метод за паралелно пресмятане на факториели и суми

Предизвикателството при паралелното пресмятане на редове от типа на Тейлър се състои в последователната зависимост на факториелите. В проекта използвам декомпозиция на домейна, при която изчислителният обхват $[0, N]$ се разделя на интервали. Паралелизмът се реализира чрез конвейерна обработка, при която всяка нишка предава междинния си резултат на следващата, след като е завършила пресмятането на своя интервал. Този подход минимизира конфликтите при достъп до паметта [2].

2.2 Метод за балансиране на задачите

За да осигурия равномерно натоварване на нишките в конвейерната архитектура, ще разгледаме тези два основни подхода: предварителна оценка на изчислителния обхват и адаптивно разпределение на задачите спрямо сложността им.

Оценка на обхвата чрез апроксимация на Стирлинг

Апроксимацията на Стирлинг е фундаментална за проекта, тъй като позволява предварително определяне на броя членове N в реда на Тейлър, необходими за постигане на желаната точност от p десетични знака. За постигане на точност от p десетични знака е необходимо да определим броя членове N така, че остатъкът $R_N \approx \frac{1}{(N+1)!}$ да бъде по-малък от 10^{-p} . За да бъде остатъкът по-малък от 10^{-p} , прилагаме логаритмична трансформация върху знаменателя:

$$\ln((N+1)!) > p \cdot \ln(10)$$

Използвайки формулата на Стирлинг $\ln(n!) \approx n \ln n - n$, неравенството се трансформира до:

$$(N+1) \ln(N+1) - (N+1) \approx p \ln(10)$$

При големи стойности на N , пренебрегвайки линейните членове спрямо логаритмичните, получаваме:

$$N \ln N \approx p \ln 10$$

Чрез преминаване към десетичен логаритъм и допълнителни алгебрични преобразувания, достигаме до практическата формула за планиране на работния диапазон:

$$N \approx \frac{p}{\log_{10} p}$$

Тъй като изчисляването на факториел с голяма стойност е изчислително много скъпо, тази оценка е критична за дефиниране на минимално необходимата горна граница на изчисленията.

Динамична декомпозиция с тежести

Тъй като сложността на изчисление на факториелите в реда на Тейлър расте нелинейно с всеки следващ член, линейното разделяне на обхвата би довело до сериозен дисбаланс — нишките, обработващи интервали с по-високи индекси, биха били значително по-натоварени.

За да се компенсира този ефект, използвам следния метод за разпределение, базиран на намаляващи тежести. Изчислителният обхват $[0, N]$ се разделя на $C = T \times I$ части (където T е броят нишки, а I — грануларността). На всяка част i се присвоява тежест W_i , която намалява линейно с нарастването на индекса:

$$W_i = C - i \quad \text{за } i \in [0, C - 1]$$

Общият размер на всеки интервал се определя пропорционално на неговата тежест спрямо общата сума на тежестите:

$$\text{Размер}_i = \max \left(1, \left\lfloor \frac{W_i}{\sum W} \cdot N \right\rfloor \right)$$

Този подход гарантира, че началните интервали (където факториелите са малки и се изчисляват бързо) съдържат повече членове, докато крайните интервали (които са изчислително много "скъпи") обхващат по-малък брой членове. Допълнително, чрез параметъра за грануларност I , разпределението може да се настройва фино, което допълнително оптимизира балансирането на натоварването и минимизира времето за престой (idle time) на нишките в конвейера.

2.3 Сложност на аритметиката с произволна точност

При изчисляване на математически константи с хиляди или милиони десетични знаци, основното изчислително време се измества от самото обхождане на цикъла към сложността на аритметичните операции. С нарастването на индекса в реда на Тейлър, локалните числители и знаменатели достигат огромни размери [4].

В конвейерния модел, междинните пресмятания в отделните етапи използват целочислена аритметика. Въпреки че съвременното умножение на големи числа е алгоритмично оптимизирано, финалното деление на акумулирания числител и знаменател изисква алгоритми за произволна точност. Забавянето при тази последна стъпка може да се превърне в тясно място (bottleneck), ако паралелизмът не е съобразен с нарастващата цена на самите аритметични операции.

2.4 Последователност на изпълнение на алгоритъма

За да се проследи цялостният жизнен цикъл на изчислителния процес, изпълнението на алгоритъма може да бъде представено в строга хронологична последователност.

Процесът стартира с фазата на подготовка, при която се изчислява общият брой необходими членове на реда на Тейлър чрез апроксимацията на Стирлинг. Веднага след това се прилага алгоритъмът за декомпозиция с намаляващи тежести, който разделя изчислителния домейн на оптимално балансирани интервали.

Втората стъпка е инициализацията на конвейерната структура. Създават се комуникационните канали (опашки), свързващи отделните етапи. Работните нишки се инстанцират и на всяка се присвоява циклично подмножество от подготвените интервали за обработка.

Следва същинската стартова фаза. Главният процес поставя базовата начална стойност в първата опашка на конвейера и подава сигнал за едновременно стартиране на всички работни процеси (нишки).

По време на самото изпълнение, всяка нишка пресмята локално частичния числител и знаменател за текущия си интервал. Когато приключи, тя блокира и изчаква получаването на натрупаната сума от предходния етап през съответната входна опашка. След получаването на данните, нишката ги комбинира с локалните си резултати чрез бърза целочислена аритметика и предава новата стойност нататък по веригата.

Алгоритъмът приключва с финализираща фаза. Когато междинните данни достигнат до последния етап на конвейера, се извършва реалното деление с произволна точност на вече акумулираните огромни числа. Генерира се финалното десетично представяне на константата e , което се връща към главния процес за запис в изходния файл.

2.5 Функционален анализ

Проектът реализира високопрецизно пресмятане на математическата константа e чрез развитието ѝ в ред на Тейлър. Тъй като пресмятането изисква работа с огромни числа, паралелният модел трябва да адресира не само разпределението на задачите, но и комуникационните разходи между нишките. За избор на оптимална архитектура са анализирани следните литературни образци с аналогична функционалност:

Метод на двоично разделяне (Binary Splitting)

Изследванията в [1] и [3] разглеждат метода на двоично разделяне като стандарт за пресмятане на редове от рационални числа с висока точност. Алгоритъмът използва декомпозиция от тип „разделяй и владей“ (Divide and Conquer), изграждайки рекурсивно дърво от частични суми. Този подход балансира големината на операндите и силно оптимизира аритметичните операции, но изисква дървовидна синхронизация на данните на всяко ниво от рекурсията, което усложнява паралелния контрол при ограничен брой нишки. Този метод е по-подходящ за архитектури с голям брой процесори.

Балансиран конвейерен паралелизъм (Load-balanced Pipeline)

В [2] се анализира изграждането на паралелни конвейери с балансирано натоварване. Там описват потокова архитектура, при която изчислителният домейн се разделя на етапи, свързани чрез буферирани опашки за предаване на съобщения. Според изследването, за да се избегне изчакването на конвейера, е ключово разпределението на тежестта да бъде съобразено със сложността на всеки етап, което вдъхновява използваната в моя проект статична декомпозиция с тежести.

Оценка на елементарни функции с произволна точност

[4] се фокусира върху същинския изчислителен проблем при работа с големи числа. Авторът доказва, че при хиляди и милиони итерации, времето за изпълнение е доминирано от огромните умножения и деления в паметта, а не от самия цикъл. Това налага използването на локални целочислени оптимизации (натрупване на локален знаменател) преди финалното скъпо деление, подход, имплементиран като защитен механизъм в линейния конвейер на моя проект.

Таблица 1: Сравнителен анализ на паралелните архитектури

Образец	Максима- лен паралели- зъм	Синхрон- ност	Тип декомпо- зиция	Баланси- ране	Гранулар- ност	Контекс- тни параметри
Моят проект	Ограничен от бр. интервали	Локално- синхронен	Линеен конвейер	Статично	Променлива	Брой членове и Десетична точност
Binary Splitting [1, 3]	Много висок ($O(N)$ в листата)	Синхронен	Дървовидна	Статично	Фина	Дълбочина на рекурсията
Load- Balanced Pipeline [2]	Среден (според дължината)	Асинхронен	Потокова	Динамично / Статично	Средна до Едра	Брой етапи в конвейера

2.6 Нефункционален анализ (Технологичен анализ)

При *метода на двоично разделяне* [1, 3] традиционно се използват езици от ниско ниво (като C/C++) в комбинация с високооптимизирани библиотеки за аритметика с произволна точност (напр. GMP - GNU Multiple Precision). Изпълнението се реализира върху мултипроцесорни платформи със споделена памет (Shared Memory), като синхронизацията по нивата на рекурсивното дърво изисква строг контрол чрез бариери и общи променливи.

При *конвейерните архитектури* [2] обикновено се залага на модели с предаване на съобщения (Message Passing) или буфери в споделената памет. Тези архитектури често се реализират чрез пулове от нишки (Thread Pools) в Java или C++, като фокусът е върху непрекъснатия поток на данните.

3 Проектиране и реализация

3.1 Функционално проектиране

3.1.1 Описание на функциите и коментар на алгоритмите

Имплементацията на проекта е направена на езика Python. Проектът е разделен на няколко основни логически модула (функции и класове), които реализират паралелното изчисление в конкретните файлове на проекта:

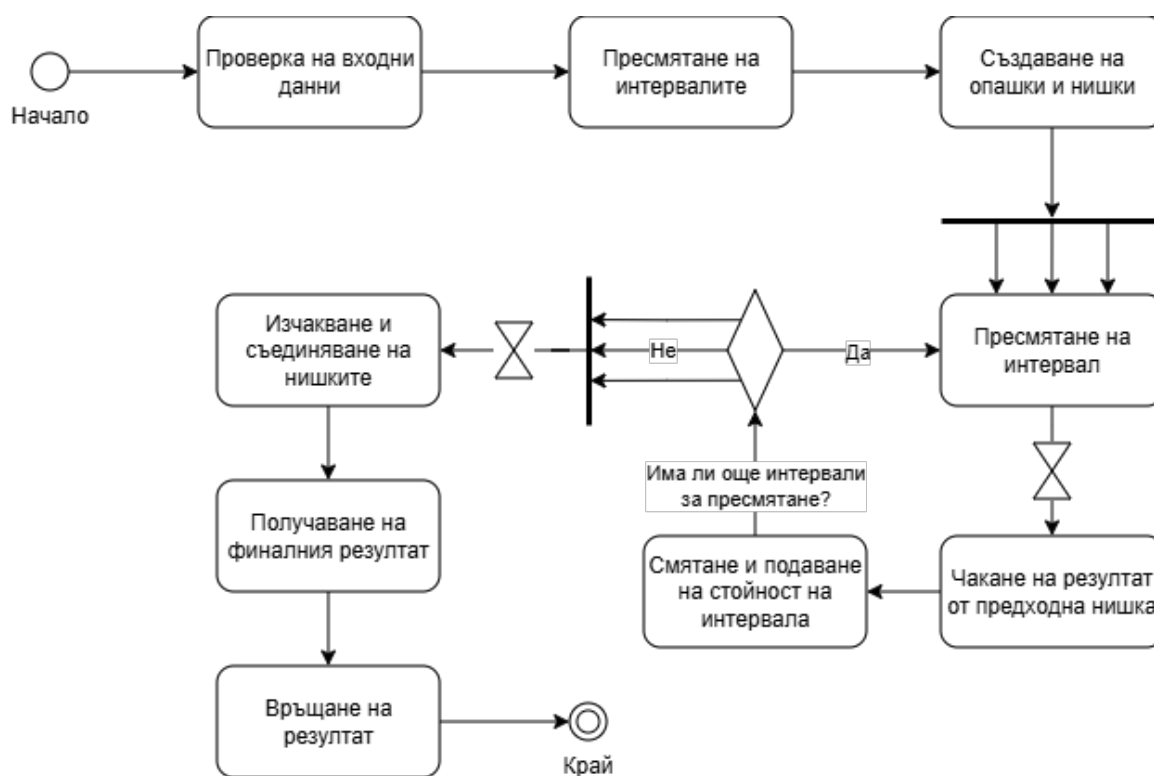
- **Спомагателни математически функции (helper.py):**
Функцията `estimate_terms(precision)` изчислява нужния брой членове на реда чрез формулата на Стирлинг спрямо желаната точност. Функцията `distribute_work(terms, threads, interval)` осъществява декомпозицията. Тя прилага алгоритъма с намаляващи тежести, за да върне списък от кортежи (интервали), които балансират изчислителната тежест.
- **Управление на конвейера (main.py):**
Функцията `setup()` инициализира конфигурацията: задава контекста за прецизност (чрез `decimal`), генерира масив от `Queue` обекти (по една за всяка граница на интервал) и инстанцира обектите `Worker`. Функцията `run(workers, queues)` оркестрира конвейера, като поставя базовата начална стойност (1.0) в първата опашка, стартира всички нишки (`start()`), изчаква тяхното приключване (`join()`) и извлича крайния резултат от последната опашка.

- **Работни нишки и синхронизация (клас `Worker` в `worker.py`):**

Всяка нишка изпълнява предефиниран `run()` метод. Алгоритъмът вътре обхожда назначените интервали циклично (чрез стъпка `threads_count`), което позволява на една нишка да участва в няколко етапа на конвейера. За текущия интервал се изчисляват локални числител и знаменател. Критичната синхронизация се случва, когато нишката извика `get()` върху съответната входна опашка, блокирайки до получаване на предходната сума. Резултатът се комбинира чрез целочислена аритметика и се изпраща (`put()`) в следващата опашка. При достигане на последния интервал, алгоритъмът завършва с реално деление на големи числа и връща крайното десетично число e .

3.1.2 Диаграми

Следните UML диаграми представят процесите в проекта:



Фигура 1: UML Activity diagram

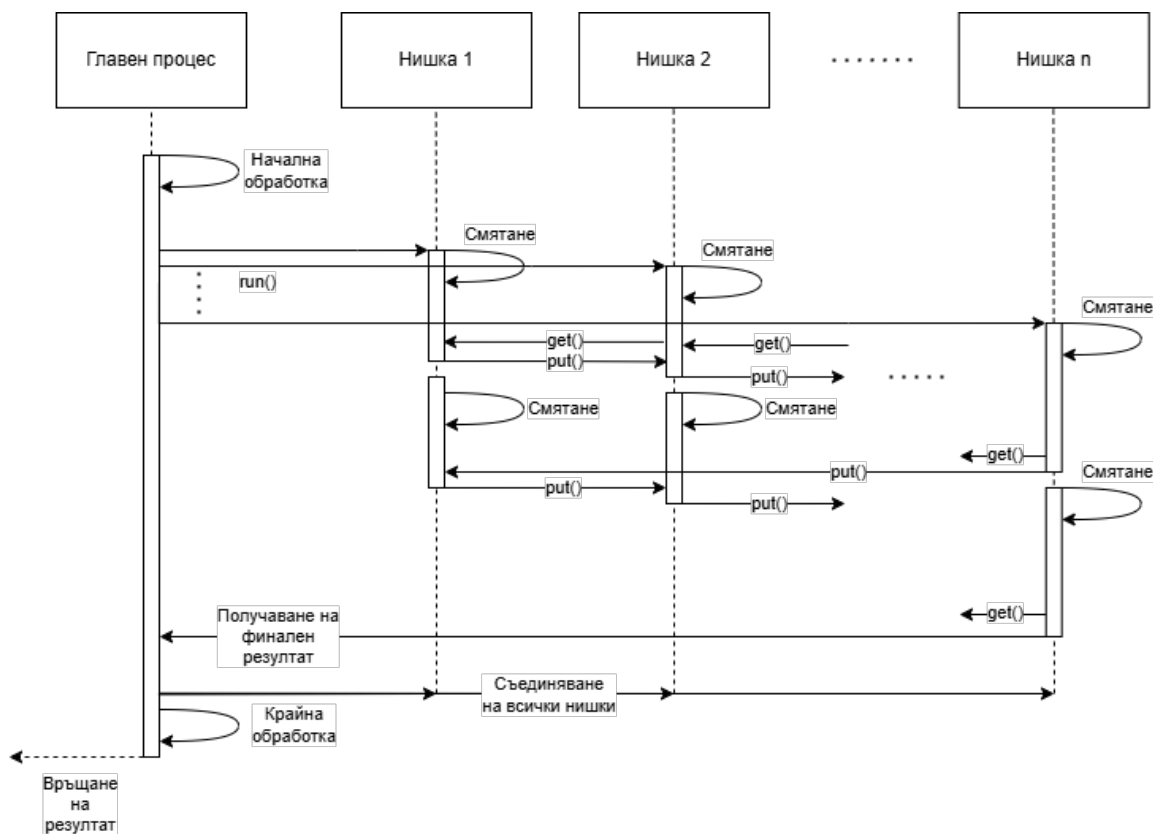
3.1.3 Потребителски интерфейси и екрани на реакция

Тъй като приложението е конзолно (CLI), интерфейсите "екрани" представляват текстови команди, а реакцията на системата — форматиран изход в терминала.

Интерфейсен екран:

Потребителят подава заявка към системата чрез терминала. Пример за стартиране с 4 нишки, точност 1000 знака и грануларност 2:

```
> python main.py -p 1000 -t 4 -i 2 result.txt
```



Фигура 2: UML Sequence diagram

Екран от реакцията на приложението:

Системата извежда детайлна информация за конфигурацията на конвейера, разпределението на интервалите и времето за изпълнение:

```

[INFO] Calculating e to 1000 decimal places.
[INFO] Taylor series terms: 333
[INFO] Using 4 worker(s) in a linear pipeline.
[INFO] Intervals per thread: 2 (8 stages total, ~41.6 terms/stage)
[INFO] Work intervals: (0, 74), (74, 138), (138, 193), (193, 239), (239, 276), ... (321, 333)
Setup time:      232600
Calculation time: 1845300
Total time:      2321100
Result written to: result.txt
  
```

Пресметнат резултат в result.txt:

```

2.71828182845904523536028747135266249775724709369995957496696762772407663035354
759457138217852516642742746639193200305992181741359662904357290033429526059563
073813232862794349076323382988075319525101901157383418793070215408914993488416
750924476146066808226480016847741185374234544243710753907774499206955170276183
860626133138458300075204493382656029760673711320070932870912744374704723069697
720931014169283681902551510865746377211125238978442505695369677078544996996794
  
```



```

686445490598793163688923009879312773617821542499922957635148220826989519366803
318252886939849646510582093923982948879332036250944311730123819706841614039701
983767932068328237646480429531180232878250981945581530175671736133206981125070
560313587516001613184717783583762166154671260638418179159281323571613203575519
282122660522466683909395375715018336334299820805243266556518634225525138672000
220108379511301277236031056342019838374673763229511069256601273041908629018150
14260965649938314236108474525189050475806519053471982322251659465314645190

```

3.1.4 Ръководство за потребителя (Параметри)

Системата изисква инсталирана версия на Python 3.14 (Free-Threading/NoGIL), която е необходима за постигане на истинско паралелно изпълнение.

Синтаксис и аргументи:

- **-t, -threads** (опционален): Брой нишки (по подразбиране 1).
- **-p, -precision** (задължителен): Брой десетични знаци.
- **-i, -interval** (опционален): Коефициент на грануларност.
- **-q, -quiet** (флаг): Потиска информационния изход.
- **file** (позиционен аргумент): Път до изходния файл.

3.2 Технологично проектиране

За нуждите на разработката, тестването и анализа на скалируемостта е използвана следната конфигурация:

Категория	Параметър	Стойност
CPU	Модел	Intel Core i7-13700K
	Архитектура	x86_64
	Общо ядра	16 (8 Р-ядра, 8 Е-ядра)
	Нишки	24
	Честота	3.4 GHz (Base) / 5.4 GHz (Turbo)
	Кеш (L1i/L1d/L2/L3)	768 KB / 640 KB / 24 MB / 30 MB
RAM	Размер	64 GB
OS	Версия	Windows 11

4 Тестване

4.1 Методология на тестване

За оценка на производителността и скалируемостта на реализираната система е разработен тестов план, базиран на следните ключови параметри, използвани за анализ на паралелното изпълнение:

- **n (точност):** Брой десетични знаци, до които се пресмята числото e . Този параметър директно определя сложността на изчислението и броя на членовете от реда на Тейлър.

- g (**грануларност**): Коефициент на сегментиране (интервали на нишка), който дефинира фиността на разпределението на изчислителните задачи.
- $T_p(i)$: Измерено време за изпълнение на конкретен опит (тест) с номер i при използване на зададен брой нишки p .
- $T_p = \min(T_p(i))$: Най-доброто (минимално) постигнато време измежду всички проведени опити за конкретна конфигурация. Тази стойност служи като база за изчисляване на показателите за производителност.
- $S_p = T_1/T_p$: Ускорение, показващо съотношението между времето за последователно изпълнение (T_1) и най-доброто време за паралелно изпълнение с p нишки (T_p).
- $E_p = S_p/p$: Ефективност, представяща колко ефективно се използват наетите ресурси (нишки) в сравнение с идеалното ускорение.

Всички времена са в наносекунди.

4.2 Резултати

- Точност 500,000 десетични знака (87735 члена от реда на Тейлър):

Таблица 2: Резултати за едра грануларност (1 интервал на нишка)

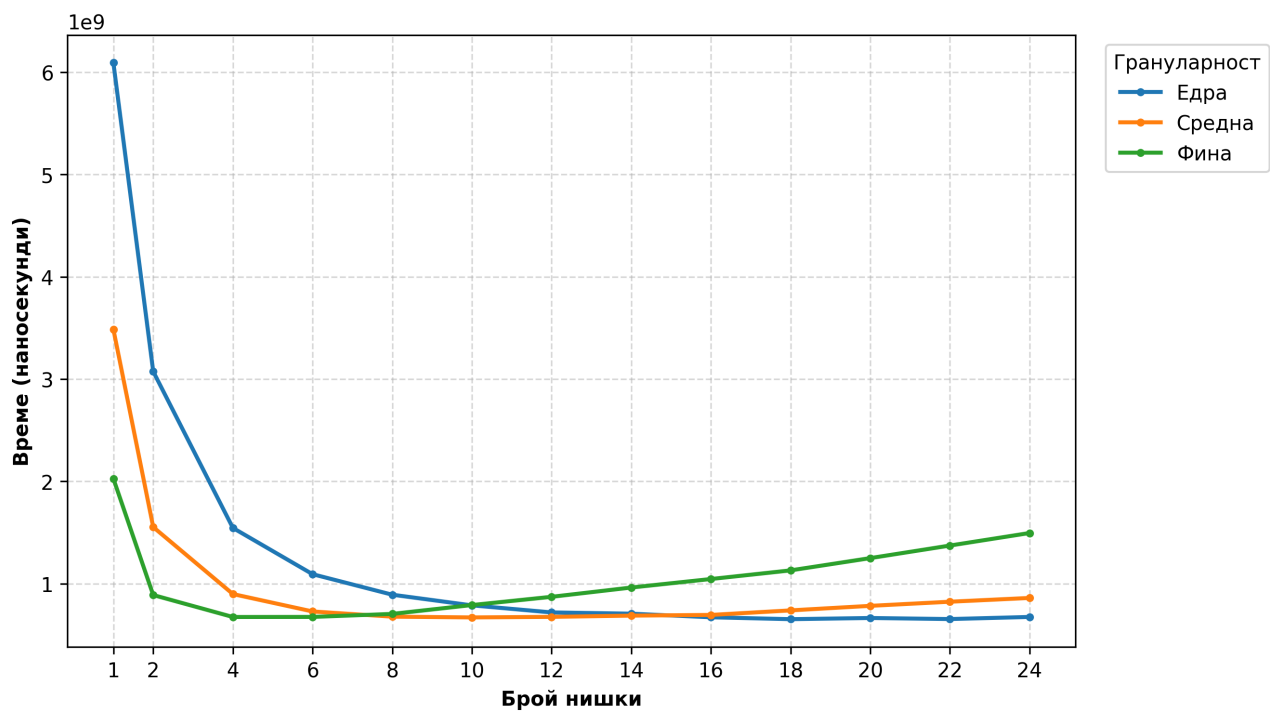
p	$T_p(1)$	$T_p(2)$	$T_p(3)$	T_p	S_p	E_p
1	6111991200	6091998800	6123987700	6091998800	1	1
2	3078790500	3074089600	3085470900	3074089600	1.981725	0.990862
4	1545577100	1544573600	1550982200	1544573600	3.944130	0.986032
6	1094811300	1103331500	1092500800	1092500800	5.576196	0.929366
8	892442700	891880400	893031500	891880400	6.830511	0.853814
10	792361100	787661500	788375200	787661500	7.734285	0.773429
12	733869800	725673700	718742400	718742400	8.475914	0.706326
14	714486500	706534000	709521000	706534000	8.622372	0.615884
16	676180800	674582800	671087700	671087700	9.077798	0.567362
18	657083700	652599800	655060000	652599800	9.334969	0.518609
20	671877700	664232600	671356800	664232600	9.171484	0.458574
22	653732000	658436900	654874300	653732000	9.318802	0.423582
24	675659800	675083500	674535500	674535500	9.031398	0.376308

Таблица 3: Резултати за средна грануларност (2 интервала на нишка)

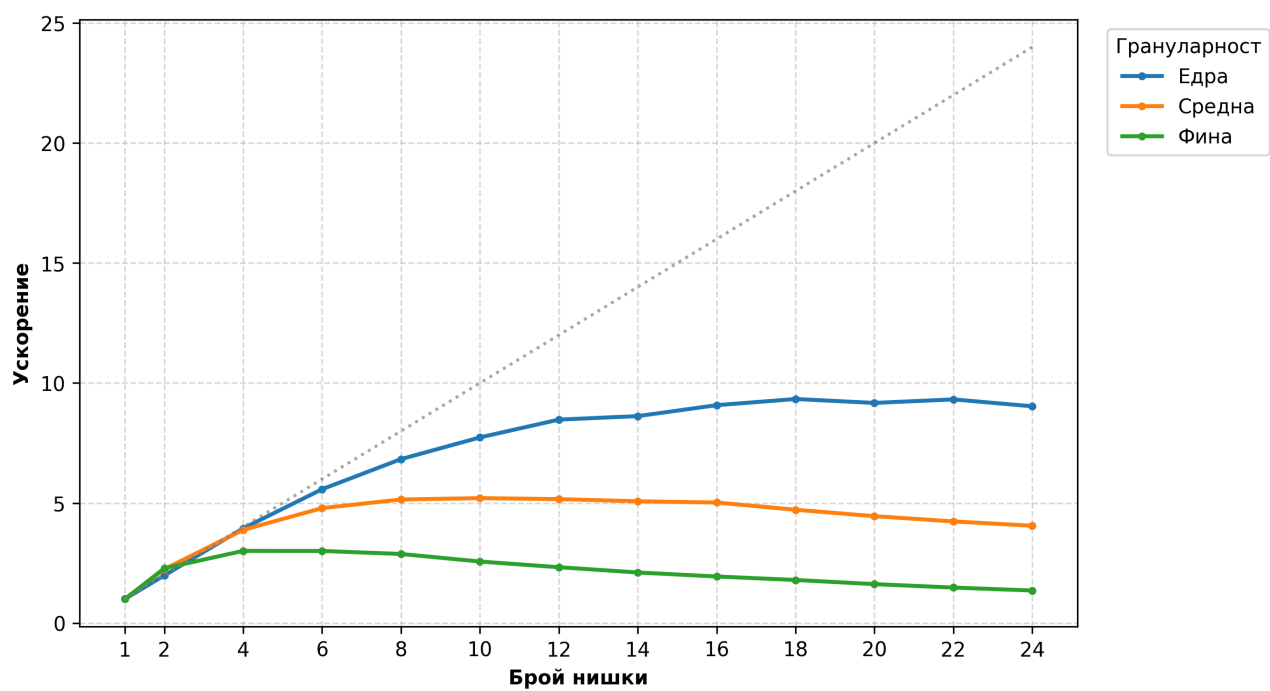
р	Тр(1)	Тр(2)	Тр(3)	Тр	Sp	Ep
1	3540503900	3486271100	3515395100	3486271100	1	1
2	1561772900	1552945800	1555718200	1552945800	2.244941	1.122470
4	899666800	899701700	898276900	898276900	3.881065	0.970266
6	728073000	731112900	733653600	728073000	4.788354	0.798059
8	687114700	683092700	677202500	677202500	5.148048	0.643506
10	670848800	678920300	669894000	669894000	5.204213	0.520421
12	678527800	675274500	682454900	675274500	5.162747	0.430229
14	699642400	687383500	699701000	687383500	5.071799	0.362271
16	705203700	702593400	694182800	694182800	5.022123	0.313883
18	743669100	739507000	738568500	738568500	4.720308	0.262239
20	783245300	783444300	782923400	782923400	4.452889	0.222644
22	824492300	829468900	823368600	823368600	4.234156	0.192462
24	861041700	864652800	860212400	860212400	4.052803	0.168867

Таблица 4: Резултати за фина грануларност (4 интервала на нишка)

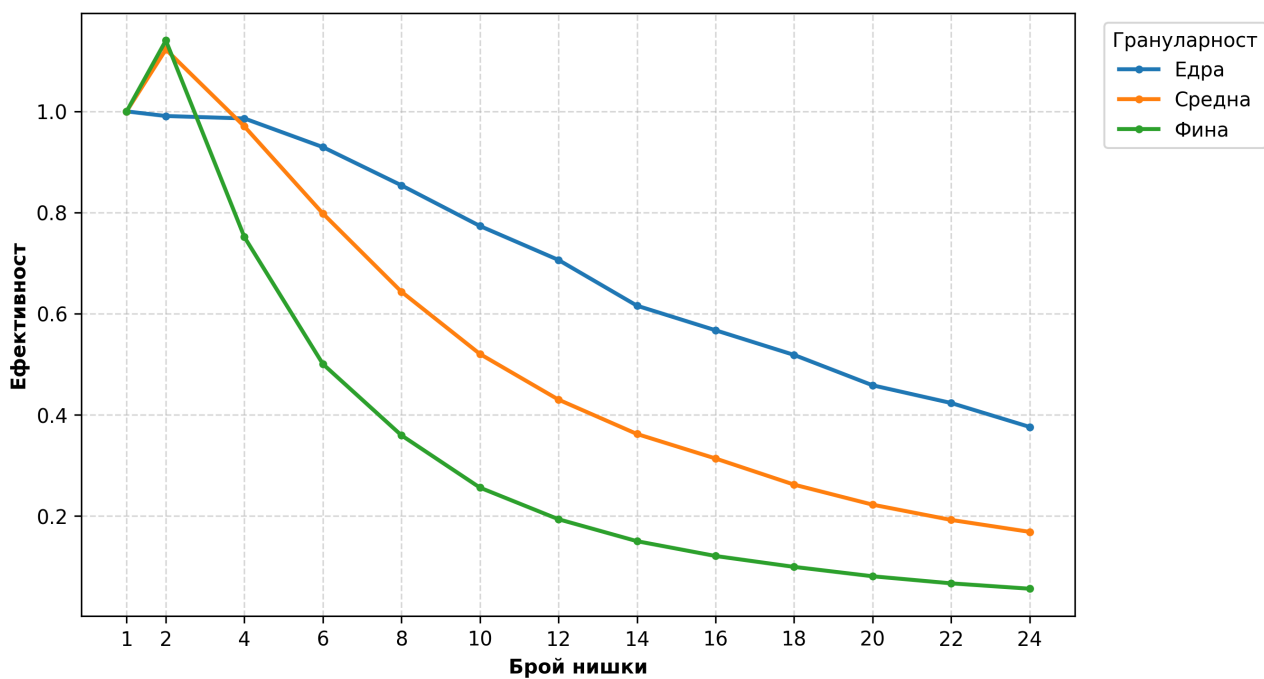
р	Тр(1)	Тр(2)	Тр(3)	Тр	Sp	Ep
1	2027282500	2037602100	2026255800	2026255800	1	1
2	891764400	888600500	888949500	888600500	2.280278	1.140139
4	676360000	675008200	673951800	673951800	3.006529	0.751632
6	674688500	677422200	676872000	674688500	3.003246	0.500541
8	704632000	714732500	704159400	704159400	2.877553	0.359694
10	799123800	790755400	802682900	790755400	2.562431	0.256243
12	872226800	871556500	871318100	871318100	2.325506	0.193792
14	973283400	962060700	966599000	962060700	2.106162	0.150440
16	1052031200	1046127900	1045267600	1045267600	1.938504	0.121157
18	1133559200	1129743200	1133052400	1129743200	1.793554	0.099642
20	1275243600	1252431000	1249666100	1249666100	1.621438	0.081072
22	1381799000	1378541300	1372529800	1372529800	1.476293	0.067104
24	1501563400	1495151100	1502203100	1495151100	1.355218	0.056467



Фигура 3: Време за изпълнение



Фигура 4: Ускорение



Фигура 5: Ефективност

- Точност 1,000,000 десетични знака (166667 члена от реда на Тейлър):

Таблица 5: Резултати за едра грануларност (1 интервал на нишка)

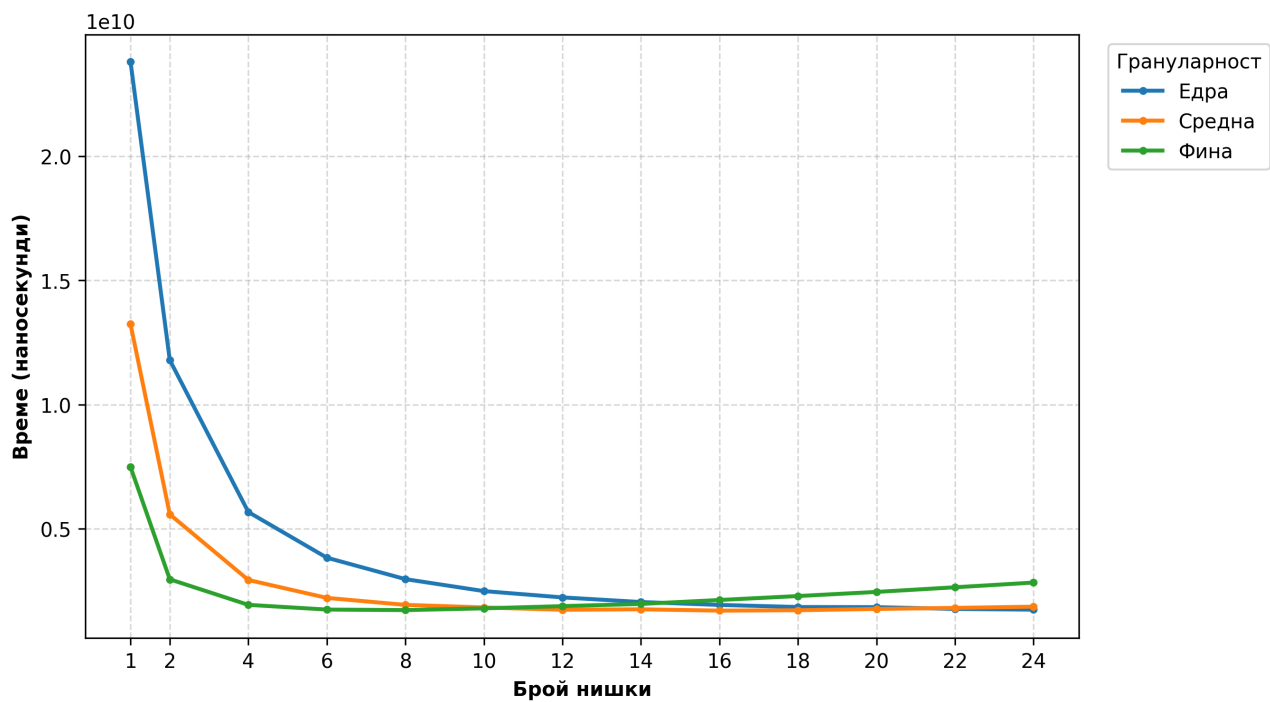
р	Тр(1)	Тр(2)	Тр(3)	Тр	Sp	Ер
1	23815861500	23799538300	23834644100	23799538300	1	1
2	11877820800	11820451500	11790694500	11790694500	2.018502	1.009251
4	5673146300	5705584200	5731917500	5673146300	4.195122	1.048780
6	3833083100	3899436500	3888566000	3833083100	6.208981	1.034830
8	3001970600	2989023700	2969742700	2969742700	8.014007	1.001751
10	2538911500	2487803200	2558077500	2487803200	9.566488	0.956649
12	2242079500	2233825600	2260313200	2233825600	10.654161	0.887847
14	2068945000	2058142700	2047923500	2047923500	11.621302	0.830093
16	1938511900	1934354300	1930755500	1930755500	12.326542	0.770409
18	1852377500	1850636700	1846948300	1846948300	12.885871	0.715882
20	1845792500	1842070600	1846197000	1842070600	12.919992	0.646000
22	1777769500	1769473900	1777244900	1769473900	13.450065	0.611367
24	1739986400	1742851000	1743930200	1739986400	13.678002	0.569917

Таблица 6: Резултати за средна грануларност (2 интервала на нишка)

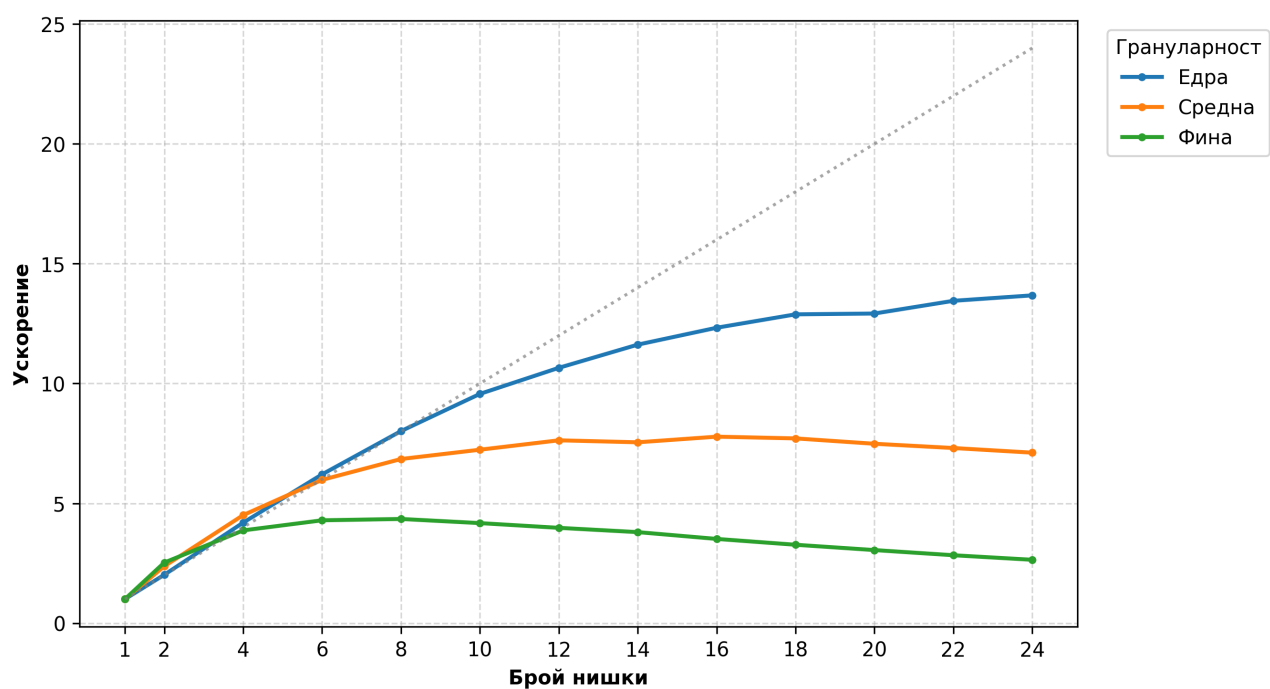
р	Тр(1)	Тр(2)	Тр(3)	Тр	Sp	Ep
1	13238472000	13272134500	13256378500	13238472000	1	1
2	5587669100	5587707000	5577579700	5577579700	2.373516	1.186758
4	2936225400	2946385800	2936989700	2936225400	4.508670	1.127168
6	2214135900	2249403100	2253664700	2214135900	5.979069	0.996512
8	1941118100	1932975600	1940818500	1932975600	6.848753	0.856094
10	1860036000	1849747500	1829299200	1829299200	7.236909	0.723691
12	1756817100	1735302500	1749159400	1735302500	7.628913	0.635743
14	1769435800	1769771300	1754379100	1754379100	7.545959	0.538997
16	1733240400	1727400900	1701370300	1701370300	7.781064	0.486317
18	1723237300	1717309500	1721915400	1717309500	7.708845	0.428269
20	1780160900	1768719300	1770863200	1768719300	7.484778	0.374239
22	1827150700	1817605300	1811641800	1811641800	7.307445	0.332157
24	1876175300	1861581100	1861037200	1861037200	7.113491	0.296395

Таблица 7: Резултати за фина грануларност (4 интервала на нишка)

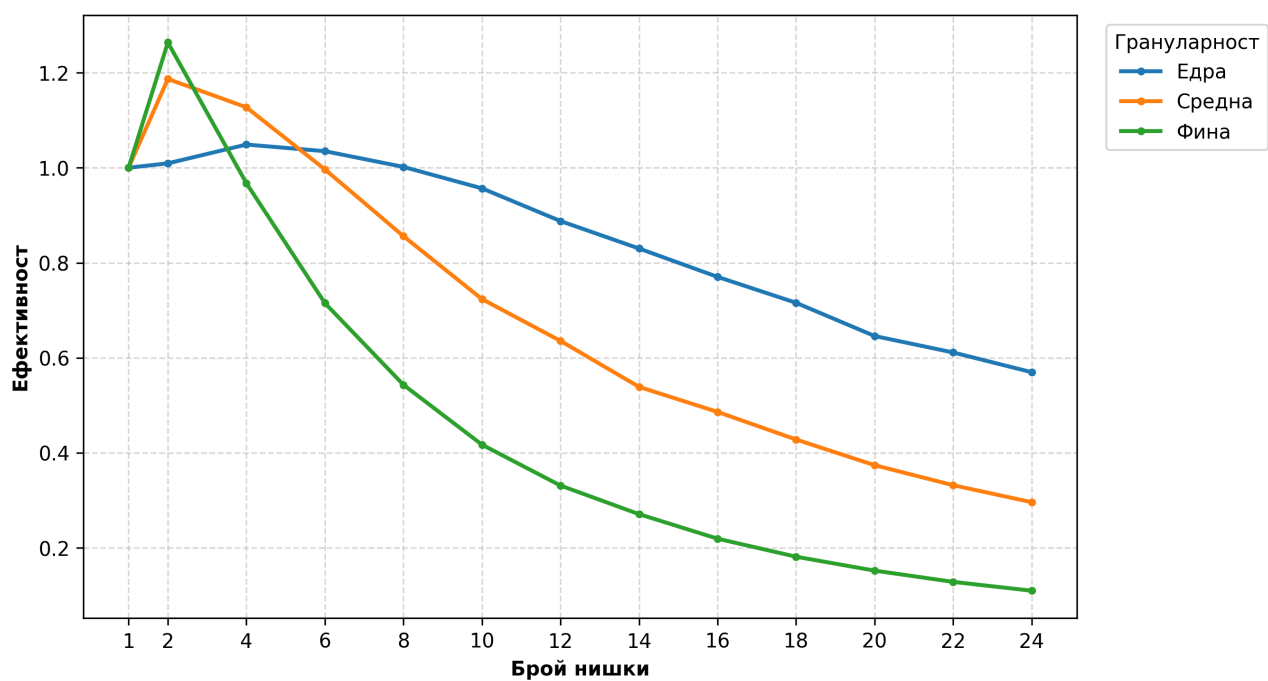
р	Тр(1)	Тр(2)	Тр(3)	Тр	Sp	Ep
1	7516540100	7484716700	7521016200	7484716700	1	1
2	2974976900	2972888100	2961928800	2961928800	2.526974	1.263487
4	1934020700	1948157700	1951988500	1934020700	3.870029	0.967507
6	1744171900	1748081800	1751430300	1744171900	4.291272	0.715212
8	1725397800	1725580800	1721889200	1721889200	4.346805	0.543351
10	1793420900	1795441000	1793178300	1793178300	4.173995	0.417399
12	1894572300	1888603100	1882231600	1882231600	3.976512	0.331376
14	1978806000	1970882300	1972199600	1970882300	3.797648	0.271261
16	2134267000	2131476400	2129924400	2129924400	3.514076	0.219630
18	2298398100	2289739100	2288588900	2288588900	3.270450	0.181692
20	2471275600	2455107800	2466941100	2455107800	3.048631	0.152432
22	2641745900	2644110900	2659459000	2641745900	2.833246	0.128784
24	2831324400	2833483700	2832252300	2831324400	2.643539	0.110147



Фигура 6: Време за изпълнение



Фигура 7: Ускорение



Фигура 8: Ефективност

4.3 Анализ и обобщение

На диаграмите се вижда, че при

5 Източници

- [1] B. Haible, & T. Papanikolaou (1997). *Fast multiprecision evaluation of series of rational numbers*. <https://www.ginac.de/CLN/binsplit.pdf>
- [2] M. Kamruzzaman, S. Swanson, & D. M. Tullsen (2013). *Load-balanced pipeline parallelism*. <https://scispace.com/pdf/load-balanced-pipeline-parallelism-3ad4huixe2.pdf>
- [3] H. Vestermarck (2025). *Exploring the Binary Splitting Method*. <https://www.hvks.com/Numerical/Downloads/HVE%20Exploring%20Binary%20Splitting%20Method.pdf>
- [4] R. P. Brent (1976). *Fast Multiple-Precision Evaluation of Elementary Functions*. <https://scispace.com/pdf/fast-multiple-precision-evaluation-of-elementary-functions-4ml8gpg10p.pdf>